

# Python Yazılım Mimarileri ve Proje Geliştirme Rehberi

Bir Python projesi geliştirmeye başlarken, uygulamanın ölçeğine, amacına ve bakım gereksinimlerine uygun bir yazılım mimarisi seçmek projenin başarısı için kritik öneme sahiptir. Bu belge, yazılım fikrinin oluşturulması aşamasından başlayarak kullanılacak teknolojilerin seçimi, dosya izin sistemlerinin kurgulanması, mimari yaklaşımlar ve işletim sistemi düzeyindeki farklılıklara kadar kapsamlı bir yol haritası sunmaktadır.

## 1. Yazılım Fikri Oluşturma ve Proje Planlama Adımları

- İhtiyaç Analizi ve Fikir Doğrulama:** Çözülecek problem netleştirilir. "Bu uygulama ne yapacak?" ve "Hangi platformlarda çalışacak?" soruları yanıtlanır.
- Mimari Kararı:** Uygulamanın bir komut satırı aracı (CLI), monolitik bir web uygulaması (örn. Django) veya dağıtık bir mikroservis (örn. FastAPI) olup olmayacağına karar verilir.
- Teknoloji Yığınının (Tech Stack) Seçilmesi:** Python sürümü, kullanılacak dış kütüphaneler, veritabanı (PostgreSQL, SQLite vb.) ve sürüm kontrol sistemi (Git) belirlenir.
- Geliştirme Ortamının Kurulumu:** İzole bir Python ortamı (sanal ortam - virtual environment) ve kod düzenleyici (Visual Studio Code vb.) yapılandırılmaları oluşturulur.

## 2. Python'da Yaygın Yazılım Mimarileri ve Dosya/Dizin Yapıları

Python kütüphaneleri (framework'ler) farklı tasarım desenlerini (design patterns) benimser. Geliştiricinin projeye başlarken bu izin yapılarını standartlara uygun oluşturması, takım çalışmasını ve kodun sürdürülebilirliğini artırır.

### A. Monolitik MVT (Model-View-Template) Mimarisi (Örn: Django)

Kapsamlı ve veri güdümlü web uygulamaları için kullanılır. Django, uygulamanın her bir modülünü (app) bağımsız tutmayı hedefler ancak genel yapı monoliktir.

```
proje_kok_dizini/
├── .venv/                # Sanal ortam klasörü
├── .git/                 # Git versiyon kontrol dizini
├── .vscode/             # Visual Studio Code yapılandırmaları
    (settings.json, launch.json)
├── .gitignore            # Git'e dahil edilmeyecek dosyalar
├── requirements.txt      # Bağımlılıkların listesi (veya
pyproject.toml)
├── manage.py            # Django yönetim betiği
├── myproject/           # Proje genel ayar klasörü (Ayarlar,
URL'ler)
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   └── wsgi.py
└── myapp/               # Uygulama modülü
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py        # Veritabanı şemaları (Model)
    ├── views.py         # İş mantığı ve denetleyiciler (View)
    └── tests.py         # Birim testleri
```

## B. Temiz Mimari (Clean Architecture / Domain-Driven Design) (Örn: FastAPI)

Ölçeklenebilir, test edilebilir ve dış bağımlılıklardan (veritabanı, UI) izole edilmiş sistemler için kullanılır. Kod, etki alanı (domain), altyapı (infrastructure) ve uygulama (application) katmanlarına ayrılır.

```
fastapi_clean_project/
├── .gitignore
├── pyproject.toml        # Modern Python paket yönetimi
├── src/
|   ├── main.py          # Uygulama giriş noktası (Entrypoint)
```

```
|   |— core/                # Konfigürasyon, güvenlik, hata
yakalama
|   |— domain/             # Saf iş mantığı, varlıklar (Entities)
ve arayüzler (Interfaces)
|   |   └─ user.py
|   |— application/        # Kullanım senaryoları (Use Cases /
Services)
|   |   └─ user_service.py
|   |— infrastructure/     # Veritabanı bağlantıları, dış API
çağrılar
|   |   └─ database.py
|   └─ presentation/       # API rotaları (Endpoints /
Controllers)
|       └─ api_v1.py
└─ tests/                  # Kapsamlı test dizini
```

### 3. Proje Başlangıcında İhtiyaç Duyulan Temel Dosyalar ve Teknolojiler

| Dosya / Teknoloji                 | Açıklama ve İşlevi                                                                                                                                      |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| .gitignore                        | Derlenmiş dosyaların (__pycache__), sanal ortamların ve hassas anahtarların kaynak kod deposuna (GitHub vb.) gönderilmesini engeller.                   |
| pyproject.toml / requirements.txt | Projede kullanılan kütüphanelerin sürümlerini tanımlar. Modern projelerde PEP 518 standardı gereği pyproject.toml (Poetry veya Flit ile) tercih edilir. |
| README.md                         | Projenin ne işe yaradığını, nasıl kurulacağını ve çalıştırılacağını açıklayan ana dokümantasyon dosyasıdır.                                             |

| Dosya / Teknoloji         | Açıklama ve İşlevi                                                                                                   |
|---------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Sanal Ortam (venv)</b> | Sistem genelindeki Python kütüphaneleriyle çakışmayı önlemek için projeye özel izole bir kütüphane alanı oluşturur.  |
| .env                      | Veritabanı şifreleri, API anahtarları gibi çevresel değişkenleri (Environment Variables) güvenli bir şekilde saklar. |

## 4. Mimari Yaklaşımlara Göre Uygulama Örnekleri

### Örnek 1: Temiz Mimari Kullanarak FastAPI ile Kullanıcı Servisi

Bu örnekte bağımlılıkların tersine çevrilmesi (Dependency Injection) kullanılarak iş mantığı ile web çerçevesi birbirinden ayrılmıştır.

```
# domain/user.py (Çekirdek İş Mantığı)
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str

# application/user_service.py (Kullanım Senaryosu)
class UserService:
    def __init__(self, repository):
        self.repository = repository

    def get_user(self, user_id: int) -> User:
        return self.repository.fetch(user_id)

# presentation/api.py (Web Katmanı / FastAPI Rotaları)
from fastapi import APIRouter, Depends
from application.user_service import UserService
```

```
router = APIRouter()

# Dependency Injection ile servis sağlanıyor
def get_user_service():
    # Burada repository altyapı katmanından enjekte edilebilir
    return UserService(repository=MockDatabase())

@router.get("/users/{user_id}", response_model=User)
def read_user(user_id: int, service: UserService = Depends(get_user_service)):
    return service.get_user(user_id)
```

## 5. İşletim Sistemi Farklılıkları ve Dikkat Edilmesi Gerekenler

Python genel olarak platformdan bağımsız (cross-platform) çalışsa da, alt seviye işletim sistemi etkileşimlerinde (Windows, Linux, macOS) mimari tasarımları etkileyen önemli farklılıklar bulunur:

- **Dosya Yolları (Path Handling):** Windows işletim sisteminde dosya yolları ters eğik çizgi (\) kullanırken, UNIX tabanlı sistemler (Linux/macOS) düz eğik çizgi (/) kullanır. Bu sorunu aşmak ve işletim sistemi mimarisinden bağımsız çalışmak için standart kütüphanedeki pathlib modülü (PEP 428) kesinlikle kullanılmalıdır. Geleneksel os.path yerine nesne yönelimli yol yönetimi sağlar.
- **Sanal Ortam Aktivasyonu:**
  - Windows: .venv\Scripts\activate
  - Linux / macOS: source .venv/bin/activate
- **Çoklu İşlem (Multiprocessing):** Python'da paralel işlemler oluşturulurken, Unix tabanlı sistemler varsayılan olarak belleği daha verimli kopyalayan fork() sistem çağrısını kullanır. Windows ise fork desteklemediği için spawn() yöntemini kullanır. Bu durum, Windows üzerinde if \_\_name\_\_ == '\_\_main\_\_': koruma bloğunun kullanılmasını zorunlu kılar, aksi takdirde sonsuz döngüsel işlem oluşturma hataları meydana gelir.
- **Dosya Kodlamaları (Encoding):** Linux ve macOS varsayılan olarak UTF-8 kodlamasını kullanırken, Windows (özellikle Türkçe bölgesel ayarlarında) varsayılan olarak cp1254 gibi farklı metin kodlamaları kullanabilir. Dosya okuma/yazma işlemlerinde her zaman

`open('dosya.txt', encoding='utf-8')` şeklinde kodlama açıkça belirtilmelidir.

## 6. Resmi Kaynaklar ve Atıflar

- **Python Resmi Dokümantasyonu:** Modüller, yerleşik tipler ve PEP standartları için başvurulması gereken temel kaynak (<https://docs.python.org/3/>).
- **PEP 8 - Python Kodlama Standartları:** Yazılım mimarisini oluştururken kodun okunabilirliğini sağlamak için benimsenen resmi stil rehberi.
- **Django Dokümantasyonu (MVT):** Model-View-Template mimarisinin derinlemesine incelenmesi (<https://docs.djangoproject.com/>).
- **FastAPI Dokümantasyonu:** Modern ve asenkron web API mimarileri için (<https://fastapi.tiangolo.com/>).
- **Architecture Patterns with Python (Cosmic Python):** Python'da Domain-Driven Design (DDD) ve Temiz Mimari pratiklerini uygulamak için literatürdeki ana referanslardan biridir.

Geliştirici olarak projeye başlarken bu kılavuzdaki temel dizin yapılarını oluşturup, Git aracılığıyla versiyonlamayı başlatmak ve kullanılacak mimarinin kurallarına sadık kalmak, başarılı ve ölçeklenebilir bir yazılım yaşam döngüsünün temelini atacaktır.